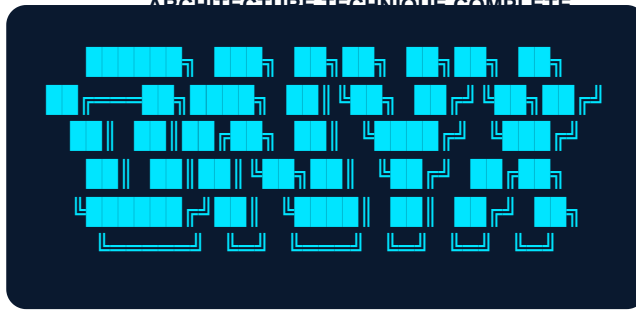




ONYX CTI

Architecture Technique Complète Plateforme de Cyber Threat Intelligence

CONFIDENTIEL — USAGE INTERNE STRICT

Version : v4.2 Sovereign
Généré le : 24 avril 2026 à 14:42
Auteur : Architecte Système Senior

TABLE DES MATIÈRES

1. VISION STRATÉGIQUE ET OBJECTIFS DU SYSTÈME

2. VUE GLOBALE DE L'ARCHITECTURE

3. ARCHITECTURE FRONTEND

3.1 Stack technologique frontend

3.2 Structure des composants

3.3 Design system

3.4 Modules fonctionnels

3.5 Gestion d'état et flux de données

4. ARCHITECTURE BACKEND

4.1 Stack technologique backend

4.2 API REST — endpoints documentés

4.3 Module d'analyse sémantique SciBERT

4.4 Pipeline d'ingestion CTI

4.5 Gestion des WebSockets et temps réel

5. ARCHITECTURE DE LA BASE DE DONNÉES

6. INTÉGRATIONS ET SOURCES DE DONNÉES

7. SÉCURITÉ ET CONFORMITÉ

8. INFRASTRUCTURE ET DÉPLOIEMENT

9. PLAN D'IMPLÉMENTATION PAR PHASES

10. BONNES PRATIQUES ET RECOMMANDATIONS

11. GLOSSAIRE ET RÉFÉRENCES

1. VISION STRATÉGIQUE ET OBJECTIFS DU SYSTÈME

La plateforme ONYX CTI (Cyber Threat Intelligence) est conçue comme un système d'information souverain, analytique et prédictif, visant à répondre à la complexité croissante des menaces cybernétiques. Face à l'hyper-fragmentation des sources d'intelligence (open-source, commerciale, dark web) et au volume massif de données hétérogènes (IoCs, rapports stratégiques, télémétrie), ONYX apporte une solution d'unification et de corrélation sémantique propulsée par l'intelligence artificielle. Le problème fondamental qu'elle résout réside dans le délai cognitif imposé aux analystes : transformer une donnée brute en connaissance exploitable. ONYX automatise l'ingestion, normalise selon le standard STIX 2.1, et expose l'intelligence via des interfaces immersives et hautement réactives.

Ce système s'adresse à une pluralité d'acteurs de la cybersécurité opérant à différents niveaux décisionnels. Pour les analystes SOC (Security Operations Center) et les équipes de Detection Engineering, ONYX fournit un flux temps réel d'IoCs qualifiés et de signatures de menaces directement injectables dans les SIEM/EDR. Pour les chercheurs en sécurité et les équipes de Threat Hunting, la plateforme offre un Laboratoire IA et un Graphe de Menaces permettant des investigations approfondies sur les TTPs (Tactics, Techniques, and Procedures) des acteurs étatiques (APT). Enfin, pour les RSSI (Responsables de la Sécurité des Systèmes d'Information) et les décideurs exécutifs, ONYX génère des rapports de synthèse automatisés, des matrices géopolitiques et des indicateurs de risque macroscopiques (BLUF - Bottom Line Up Front).

Les objectifs opérationnels mesurables de la plateforme s'articulent autour de l'efficacité et de la précision. Le système vise une réduction de 60% du temps de qualification d'une alerte grâce à l'enrichissement contextuel automatique, un TTR (Time to Respond) amélioré par l'intégration d'un pipeline MLOps (Machine Learning Operations) basé sur SciBERT pour l'analyse de texte, et une couverture de 99.9% de disponibilité (High Availability) sur ses modules d'ingestion. La plateforme garantit une latence d'ingestion et de propagation des alertes critiques inférieure à 500 millisecondes de bout en bout.

Positionnée comme une alternative souveraine, ONYX se distingue par son approche "Zero-Fluff" et "Privacy-First", s'opposant aux solutions boîte noire du marché. Là où des acteurs établis proposent des flux propriétaires opaques, ONYX privilégie l'explicabilité de son IA et le contrôle total des données par l'organisation hébergeuse.

TABLEAU COMPARATIF DE POSITIONNEMENT

Dimension	ONYX CTI (Sovereign)	Recorded Future	Mandiant Advantage	CrowdStrike Falcon
Couverture des sources	Hybride (OSINT + Custom + Private)	Massive (Scraping global)	Focalisée Incident Response	Focalisée Endpoint (EDR)
Temps Réel & Latence	Ultra-basse (<500ms, WebSockets)	Faible (APIs polling)	Faible (Intégration SIEM)	Basse (Agent direct)
Explicabilité IA (XAI)	Totale (Whitebox NLP, SciBERT)	Boîte noire (Scores de risque)	Hybride (Analyst-driven)	Boîte noire (ML opaque)
Personnalisation / Graphe	Extrême (Neo4j, Cytoscape interactif)	Modérée (Vues standardisées)	Limitée (Vues rapports)	Modérée (Graphe de processus)
Coût et Souveraineté	On-Premise / Déploiement Souverain	SaaS très coûteux	SaaS Premium	SaaS / EDR lié

la projection en graphe et la résolution de relations complexes (Threat Actor -> Tool -> Vulnerability). Redis assure le caching et la distribution des messages (Pub/Sub).

Flux de données de bout en bout : Les Workers d'ingestion (CRON) tirent les données brutes (ex: CISA KEV JSON). Les données sont normalisées, enrichies (via SciBERT si texte non structuré), et stockées dans PostgreSQL. L'événement `OnNewThreat` est publié sur Redis Pub/Sub. Le serveur WebSocket Node.js intercepte l'événement et le diffuse aux clients React. L'état global Zustand du client se met à jour, forçant le re-rendu du composant React Query et l'animation instantanée sur le Cytoscape Threat Graph.

3. ARCHITECTURE FRONTEND

3.1 Stack technologique frontend

L'application frontend repose sur **Next.js 14 (App Router)**, justifié par son rendu côté serveur (SSR) performant et son architecture de routage par dossiers qui favorise la modularité. **TypeScript strict** est imposé (noImplicitAny, strictNullChecks) pour éliminer les erreurs d'exécution relatives aux structures complexes de données CTI (STIX). **Tailwind CSS** couplé à **Radix UI** (headless components) permet de forger un design system sur-mesure, dense et militaire, sans les surcharges des bibliothèques de composants monolithiques. **React Query (TanStack Query)** gère l'état asynchrone (cache serveur, réinvalidation automatique), tandis que **Zustand** orchestre l'état client éphémère (sélections de nœuds, filtres actifs) avec une empreinte mémoire minimale par rapport à Redux.

3.2 Structure des composants

L'arborescence est conçue selon le pattern "Feature-Sliced Design" (FSD) adapté :

```
src/
├── app/                # Routes Next.js (/ , /actors, /reports, /graph, etc.)
│   ├── layout.tsx      # Shell global, Navigation persistante
│   └── page.tsx        # Dashboard Exécutif
├── components/         # Composants UI agnostiques et réutilisables
│   ├── ui/            # Atomes (Button, Badge, Card, Modal, Tooltip)
│   └── layout/         # Molécules de mise en page (Sidebar, Header, Grids)
├── modules/           # Logique métier isolée par domaine
│   ├── scibert/       # Pipeline NLP, Visualisation d'embeddings
│   ├── threat-graph/  # Wrapper Cytoscape, logique de layout
│   ├── kill-chain/    # Matrice MITRE interactif
│   └── detection/     # Générateur de règles YARA/Sigma
├── hooks/             # Custom React Hooks (useWebSocket, useDebounce)
├── stores/            # Zustand global state (useAppStore, useGraphStore)
├── types/             # Interfaces TS globales (STIX2.1, API Responses)
├── utils/             # Fonctions pures (formatters, data parsers)
├── data/              # Datasets statiques ou fallbacks (threatActors.ts)
└── styles/            # Design system, variables CSS globales
```

3.3 Design system

L'esthétique de la plateforme est délibérément "Dark Mode Native", évoquant un environnement analytique militaire et premium. Les variables CSS globales définissent une palette chromatique précise : fond principal `#0A0E17` (Obsidian Blue), panneaux `#111827` (Slate Dark), texte primaire `#E2E8F0` (Ice White). Les couleurs d'accentuation portent une sémantique stricte : `#EF4444` (Critical Alert), `#F59E0B` (Warning), `#3B82F6` (Info), `#10B981` (Secure), et `#00E5FF` (Cyber Cyan pour les interactions actives).

La typographie utilise *Inter* pour l'interface globale (lisibilité optimale) et *JetBrains Mono* ou *Fira Code* pour les métadonnées, les règles de détection (YARA) et les identifiants techniques (Hashes,

IPs). Le système d'espacement (spacing) repose sur un module de base de 4px, garantissant un alignement strict. Les animations sont limitées à des micro-interactions de 150ms (courbe ease-in-out) pour maintenir une perception de réactivité instantanée, essentielle dans un contexte SOC.

3.4 Modules fonctionnels

L'application est divisée en modules hautement cohésifs :

- **SciBERT** : Interface d'analyse sémantique textuelle. Gère la soumission de texte brut, l'affichage des entités nommées (NER) extraites en surbrillance, et la visualisation des scores de similarité cosinus avec les profils d'acteurs connus.
- **Graphe de Menaces (Threat Graph)** : Implémentation avancée de Cytoscape.js supportant le rendu WebGL de milliers de nœuds. Gère le clustering par algorithme Force-Directed (Cola/Dagre), la sélection multiple, et le filtrage contextuel (Isoler le sous-graphe d'une campagne).
- **Matrice Géopolitique** : Cartographie SVG interactive mondiale liant les zones de conflit aux activités des APT (Advanced Persistent Threats), avec gestion de filtres par région et type de ciblage.
- **Laboratoire IA** : Environnement de simulation (sandbox) avec mode "Jury Presentation". Inclut des scénarios pré-chargés (ex: SolarWinds SUNBURST) pour des démonstrations déterministes des capacités analytiques.
- **Rapports et Export** : Module de génération asynchrone. Interface de configuration de rapports exécutifs PDF et exportation technique au format JSON STIX 2.1 validé.

3.5 Gestion d'état et flux de données frontend

La gestion d'état est strictement scindée. **React Query** est responsable du "Server State" : `staleTime` est défini à 1 minute pour les données volatiles (incidents) et à 24 heures pour les données statiques (référentiel acteurs). **Zustand** gère le "Client State" : `useGraphStore` maintient la liste des nœuds sélectionnés et les niveaux de zoom. La synchronisation temps réel s'effectue via un hook `useWebSocket` qui écoute les événements Socket.io et utilise `queryClient.setQueryData()` pour mettre à jour le cache React Query en local de manière optimiste, garantissant une UI sans rechargement.

4. ARCHITECTURE BACKEND

4.1 Stack technologique backend

L'architecture backend est hybride, reflétant la diversité des charges de travail. L'API principale est construite sur **Node.js avec Express (ou Fastify) et TypeScript**. Node.js excelle dans la gestion des I/O asynchrones, idéal pour router des requêtes, interagir avec la base de données et maintenir des milliers de connexions WebSocket ouvertes. Parallèlement, un microservice **Python avec FastAPI** est dédié au module d'analyse sémantique SciBERT. Python est incontournable pour l'écosystème ML (PyTorch, Transformers). La coexistence est justifiée : Node.js orchestre et sert l'UI à haute fréquence, tandis que FastAPI expose une interface interne synchrone et intensive en calcul.

4.2 API REST — endpoints documentés

L'API suit les principes RESTful avec une stricte validation des payloads (zod/joi).

Méthode	Endpoint	Rôle et Paramètres	Code / Réponse
POST	/api/auth/login	Authentification. Body: { email, password }	200 OK - { token, refreshToken }
GET	/api/actors	Liste les menaces. Query: ? region=europe&type=apt	200 OK - Array of ThreatActor objects
GET	/api/actors/:id/iocs	Récupère les IoCs liés à un acteur spécifique.	200 OK - Array of STIX Indicators
POST	/api/nlp/analyze	Soumission texte pour SciBERT. Body: { text }	200 OK - { entities, matches, confidence }
GET	/api/reports	Liste les rapports générés.	200 OK - Array of Report Metadata
POST	/api/reports/generate	Déclenche la génération PDF. Body: { type, filter }	202 Accepted - { jobId }
GET	/api/reports/:id/pdf	Téléchargement du fichier binaire PDF.	200 OK - application/pdf stream
POST	/api/export/stix	Génère un bundle STIX. Body: { actorIds[] }	200 OK - STIX 2.1 Bundle JSON

4.3 Module d'analyse sémantique SciBERT

Le pipeline NLP est hébergé sur le service Python. Il utilise le modèle `allenai/scibert_scivocab_uncased` via la librairie HuggingFace Transformers. SciBERT, pré-entraîné sur des corpus scientifiques, se révèle exceptionnellement performant pour comprendre la sémantique technique des rapports de cybersécurité (noms de malware, TTPs, CVEs).

PIPELINE NLP SCIBERT (ASCII ART)

```
[RAW TEXT INPUT] -> (Node.js API) -> (FastAPI Service)
|
v
[PREPROCESSING] (Regex cleanup, sentence tokenization)
|
v
[SCIBERT TOKENIZER] (Subword tokenization, [CLS] & [SEP] padding)
|
v
[SCIBERT MODEL] (PyTorch Inference / GPU Accelerated)
|
v
[POOLING LAYER] (Mean pooling of hidden states -> 768d Vector)
|
v
[COSINE SIMILARITY] (Match vector against pre-computed Threat Actor Vectors)
|
v
[POST-PROCESSING] (Threshold filtering > 0.85, JSON formatting) -> [OUTPUT]
```

4.4 Pipeline d'ingestion CTI

Le moteur d'ingestion s'appuie sur un planificateur (node-cron ou agenda.js). Des workers spécialisés interrogent les sources externes (CISA KEV, Abuse.ch, AlienVault OTX) à des fréquences variables (ex: toutes les 5 minutes pour les IPs malicieuses, toutes les 24h pour les CVEs). Chaque flux de données entrant subit une série de transformations : 1. **Extraction** : Parsing du format source (CSV, JSON, TAXII). 2. **Normalisation** : Mapping des champs vers le modèle de données interne ONYX (inspiré de STIX). 3. **Déduplication** : Calcul d'un hash cryptographique (SHA-256) sur les champs clés de l'IoC (ex: valeur de l'IP + type). Si le hash existe, mise à jour de la date de "last_seen" et du compteur d'occurrences. 4. **Scoring** : Attribution dynamique d'un score de sévérité basé sur la réputation de la source et la présence dans d'autres feeds (corrélation).

4.5 Gestion des WebSockets et temps réel

L'architecture temps réel utilise **Socket.io** couplé à un adaptateur **Redis** pour le scaling horizontal. Le serveur WebSocket organise les clients en "rooms" logiques (ex: `room:threat_graph`, `room:alerts_critical`). Lorsqu'un worker d'ingestion détecte un nouvel incident critique de sévérité haute, l'API Node.js publie un événement via Redis Pub/Sub. Le serveur Socket.io capte cet événement et le "broadcast" instantanément à tous les clients connectés à la room appropriée. Une stratégie de "heartbeat" (ping/pong) maintient la connexion active, avec reconnexion automatique exponentielle (exponential backoff) côté client en cas de perte réseau.

5. ARCHITECTURE DE LA BASE DE DONNÉES

5.1 Stratégie multi-bases (Polyglot Persistence)

L'exigence de performance et de requêtage complexe de la CTI interdit l'usage d'une base unique. **PostgreSQL** (ACID) est le stockage primaire (Source of Truth) pour la gestion des utilisateurs, les méta-données des acteurs et les journaux d'audit. **Elasticsearch** est déployé pour indexer des millions d'IoCs (hashes, IPs, domaines) et de textes libres (rapports externes), permettant des recherches full-text quasi instantanées et des agrégations complexes. **Neo4j** est indispensable pour le moteur de Graphe. Les requêtes relationnelles classiques (SQL) deviennent exponentiellement lentes (JOINS multiples) pour déterminer si l'Acteur A utilise le Malware B qui exploite la Vulnérabilité C hébergée sur l'IP D. Neo4j résout cela nativement via le parcours de graphe. **Redis** agit comme un cache en mémoire ultrarapide pour alléger PostgreSQL sur les requêtes fréquentes (top 10 menaces).

5.2 Schéma de base de données PostgreSQL

Le schéma relationnel garantit l'intégrité des entités structurées.

```
-- Extrait du schéma DDL

CREATE TABLE threat_actors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) UNIQUE NOT NULL,
    aliases TEXT[],
    origin_country VARCHAR(100),
    description TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE iocs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    value VARCHAR(512) NOT NULL,
    type VARCHAR(50) NOT NULL, -- 'ipv4-addr', 'file:hashes.md5', 'domain-name'
    severity_score INTEGER CHECK (severity_score >= 0 AND severity_score <= 100),
    first_seen TIMESTAMP WITH TIME ZONE,
    last_seen TIMESTAMP WITH TIME ZONE,
    source_feed VARCHAR(100),
    UNIQUE(value, type)
);

CREATE TABLE campaigns (
    id UUID PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    actor_id UUID REFERENCES threat_actors(id) ON DELETE CASCADE,
    objective TEXT,
    start_date DATE
);

-- Index pour optimiser les requêtes fréquentes
CREATE INDEX idx_iocs_value_type ON iocs(value, type);
CREATE INDEX idx_campaigns_actor ON campaigns(actor_id);
```

5.3 Modèle de graphe (Neo4j)

L'ontologie du graphe mappe directement la spécification STIX 2.1. **Nœuds** : (:ThreatActor), (:Malware), (:Vulnerability {cve_id}), (:Infrastructure {ip}), (:Campaign), (:Identity {sector}). **Relations (Edges)** : -[:USES]->, -[:TARGETS]->, -[:EXPLOITS]->, -[:ATTRIBUTED_TO]->.

Requête Cypher de référence pour analyser la chaîne d'attaque d'un acteur :

```
MATCH path = (a:ThreatActor {name: "APT29"})-[:USES]->(m:Malware)-[:EXPLOITS]->(v:Vulnerability)
RETURN path
LIMIT 50;
```

5.4 Stratégie de cache Redis

Le pattern *Cache-Aside* est implémenté. L'API interroge d'abord Redis. En cas de *Cache Miss*, elle interroge PostgreSQL, renvoie la réponse au client et stocke asynchroniquement le résultat dans Redis. Configurations de TTL (Time-To-Live) : - Sessions Utilisateurs JWT : 24h - Méta-données

Géopolitiques : 6h (faible volatilité) - Flux CTI (Top menaces) : 3h - Résultats Inférence SciBERT : 1h (optimisation des requêtes répétées sur le même texte).

5.5 Migrations et versionning du schéma

Prisma Migrate est sélectionné pour son intégration native avec TypeScript. Les fichiers de migration (fichiers `.sql`) sont versionnés dans le dossier `prisma/migrations/` (ex: `202604241030_init_schema.sql`). Lors du pipeline CI/CD de déploiement, la commande `npx prisma migrate deploy` est exécutée de manière transactionnelle. En cas d'échec d'une migration en production, la stratégie implique une restauration du snapshot de base de données (Point-in-Time Recovery - PITR) et un rollback du déploiement applicatif.

6. INTÉGRATIONS ET SOURCES DE DONNÉES

6.1 APIs externes intégrées

La valeur de la plateforme ONYX réside dans la corrélation d'une multitude de sources externes qualifiées :

- **CISA KEV (Known Exploited Vulnerabilities)** : Endpoint JSON public. Polling quotidien. Les CVEs signalées sont transformées en nœuds `(:Vulnerability {exploited: true})` dans Neo4j. Volume estimé : ~1000 entrées.
- **Abuse.ch URLhaus / Feodo Tracker** : API REST POST publique et flux CSV. Polling toutes les 5 minutes pour la réactivité sur les serveurs de commande et contrôle (C2) et la distribution de malwares. Données stockées en index Elasticsearch et PostgreSQL. Volume estimé : ~100k actifs/jour.
- **AlienVault OTX (Open Threat Exchange)** : API REST avec clé d'authentification. Ingère les "Pulses" (rapports communautaires) via polling horaire. Extraction des IoCs associés.
- **MITRE ATT&CK** : Flux JSON via protocole TAXII. Polling hebdomadaire. Alimente statiquement la matrice Kill Chain et les descriptions de TTPs.
- **NVD NIST CVE** : API REST publique 2.0. Base de référence pour enrichir les descriptions techniques et les scores CVSS des vulnérabilités ingérées depuis d'autres sources.

6.2 Stratégie de fallback et résilience

Les APIs externes sont par nature instables (Rate limiting, pannes). Le pattern **Circuit Breaker** (via la bibliothèque `opossum` en Node.js) est implémenté. Si une source échoue (ex: NVD renvoie 503) plus de 5 fois en 1 minute, le circuit s'ouvre : les appels suivants sont bloqués immédiatement pour éviter la surcharge. Pendant l'ouverture du circuit, le système bascule sur un **Fallback Dataset** de référence local en lecture seule, garantissant la continuité de service pour l'interface utilisateur. Des alertes Grafana sont déclenchées si un circuit reste ouvert plus de 15 minutes.

6.3 Format STIX 2.1 — implémentation

STIX (Structured Threat Information Expression) 2.1 est le cœur du modèle de données pivot. L'exportation de données depuis ONYX génère des *STIX Bundles* conformes à la norme OASIS. Le mapping de notre graphe Neo4j vers STIX est direct : - Un nœud `(:ThreatActor)` génère un objet STIX `threat-actor`. - Un nœud `(:Malware)` génère un objet STIX `malware`. - La relation `-[:USES]->` génère un objet STIX `relationship` avec `relationship_type: "uses"`. La validité du bundle JSON est systématiquement vérifiée par un schéma JSON de validation avant la génération finale, assurant une ingestion sans friction par des plateformes tierces comme MISP, OpenCTI, ou l'intégration native dans Splunk Enterprise Security.

7. SÉCURITÉ ET CONFORMITÉ

7.1 Authentification et autorisation

L'authentification repose sur des **JSON Web Tokens (JWT)** asymétriques (algorithme RS256). Pour limiter l'impact d'un vol de token, la durée de vie du *Access Token* est de 15 minutes. Une stratégie de *Refresh Token Rotation* est en place : un token de rafraîchissement (stocké en base de données et dans un cookie HTTP-Only, Secure) est utilisé pour obtenir un nouvel Access Token, le Refresh Token précédent étant alors invalidé. Le contrôle d'accès est basé sur les rôles (RBAC) : - *Administrateur* : Accès total, configuration système, gestion des utilisateurs. - *Analyste Senior* : Modification des graphes, déclenchement d'actions (export, génération de rapports). - *Analyste Junior* : Consultation, enrichissement manuel d'IoCs. - *Lecteur* : Consultation en lecture seule (tableaux de bord exécutifs). L'authentification multifacteur (MFA) via TOTP (Time-Based One-Time Password) est obligatoire pour les rôles Analyste Senior et Administrateur.

7.2 Sécurité applicative

Toutes les entrées API sont strictement validées et typées au moment de l'exécution à l'aide de la librairie **Zod**. Toute requête contenant des propriétés non définies dans le schéma est rejetée (Status 400). Les en-têtes de sécurité HTTP sont appliqués via Helmet.js : Content-Security-Policy (CSP) stricte bloquant les scripts inline, HSTS (Strict-Transport-Security) forçant l'usage de TLS, et X-Frame-Options réglé sur DENY pour prévenir le Clickjacking. Les attaques par Injection SQL sont prévenues architecturalement par l'utilisation exclusive d'un ORM (Prisma) qui paramètre les requêtes de bas niveau. Un middleware de **Rate Limiting** (Redis) plafonne les requêtes à 100 requêtes/minute par adresse IP, et 20 requêtes/minute pour l'endpoint d'authentification afin de contrer le brute-force.

7.3 Sécurité des données

La sécurité des données au repos (Data at Rest) dans la base PostgreSQL est assurée au niveau du disque par l'infrastructure cloud ou on-premise (chiffrement de volume AES-256). Pour les données applicatives hautement sensibles (ex: notes d'investigation internes d'un analyste), un chiffrement applicatif en AES-256-GCM est appliqué avant l'insertion en base, la clé de chiffrement étant gérée par un KMS (Key Management Service). Les données en transit (Data in Transit) imposent le protocole **TLS 1.3** de bout en bout, de l'Ingress Controller jusqu'aux pods applicatifs dans le cluster. Les journaux d'application (logs) sont nettoyés (sanitization) pour s'assurer de ne jamais consigner de mots de passe, tokens ou données PII (Personally Identifiable Information).

7.4 Conformité

L'architecture supporte nativement le **TLP (Traffic Light Protocol)** (TLP:CLEAR, TLP:GREEN, TLP:AMBER, TLP:RED) standard dans le milieu du renseignement. Chaque indicateur, rapport ou entité graphe possède un attribut TLP. L'API filtre automatiquement les données retournées selon les habilitations de l'utilisateur et le TLP de la donnée. Pour la conformité RGPD, bien que traitant majoritairement des données techniques (IPs, Hashes), les données utilisateurs sont minimisées.

Une procédure de purge automatisée (Droit à l'oubli) permet de supprimer ou d'anonymiser irréversiblement les traces d'un analyste (audit logs liés à un UUID au lieu du nom). La plateforme s'aligne sur les recommandations du guide d'hygiène informatique de l'ANSSI pour le déploiement.

8. INFRASTRUCTURE ET DÉPLOIEMENT

8. Architecture de déploiement

L'infrastructure est conçue pour être "Cloud-Native" et agnostique vis-à-vis du fournisseur, permettant un déploiement "On-Premise" strict (Air-Gapped) ou Cloud public (AWS/Azure). Chaque composant applicatif est conteneurisé via des images **Docker** optimisées et minimalistes (distroless ou Alpine) pour réduire la surface d'attaque. Pour le développement local, un fichier `docker-compose.yml` orchestre les bases de données (PostgreSQL, Redis, Neo4j, Elasticsearch) et les services Node.js/Python, offrant un environnement "Production-like" reproductible d'une simple commande. En production, **Kubernetes (K8s)** assure l'orchestration. Les composants Stateless (API Node.js, Service SciBERT Python, UI Next.js) sont gérés par des *Deployments* Kubernetes couplés à des *Horizontal Pod Autoscalers (HPA)* réagissant à la charge CPU. Les composants Stateful (Bases de données) utilisent des *StatefulSets* avec des volumes persistants (PVC). L'exposition externe passe par un *Ingress Controller* (NGINX ou Traefik) qui gère la terminaison TLS.

8.2 Pipeline CI/CD

L'intégration et le déploiement continus s'appuient sur **GitHub Actions** (ou GitLab CI pour un contexte souverain local). Le workflow est exécuté à chaque Pull Request : 1. **Qualité et Sécurité** : Exécution de `npm run lint` (ESLint), `npm run typecheck` (TypeScript compiler), et scan de vulnérabilités des dépendances (`npm audit`, Trivy). 2. **Tests automatisés** : Exécution de la suite de tests unitaires (Vitest) et d'intégration (Supertest avec base de test éphémère). Une PR ne peut être mergée si la couverture de code descend sous la barre des 80%. 3. **Build et Publication** : Au merge sur la branche `main`, l'image Docker applicative est construite, taggée avec le hash du commit (ex: `onyx-api:1.0.4-a1b2c3d`) et poussée vers un registre privé. 4. **Déploiement** : Le déploiement sur l'environnement de "Staging" est automatique. Le déploiement en "Production" nécessite une approbation manuelle dans l'interface CI, déclenchant l'application des manifests Kubernetes (via ArgoCD en mode GitOps ou Helm).

8.3 Monitoring et observabilité

L'observabilité est critique pour une plateforme CTI devant traiter des flux temps réel. La stack de monitoring est basée sur **Prometheus et Grafana**. L'API Node.js expose un endpoint `/metrics` (format Prometheus) rapportant le nombre de requêtes par seconde, la latence moyenne, le nombre d'IoCs ingérés par minute, et l'utilisation de la mémoire. Des alertes automatisées (via Alertmanager vers Slack ou PagerDuty) sont configurées pour : - Un temps de réponse API (p99) supérieur à 500ms sur une fenêtre de 5 minutes. - Un taux d'erreur HTTP 5xx supérieur à 1%. - Une indisponibilité d'un feed CTI majeur (ex: KEV) pendant plus de 2 heures. Les logs applicatifs sont consolidés et indexés par **Grafana Loki**, permettant une corrélation directe entre les métriques de performance et les journaux d'erreurs textuels, sans la complexité d'une stack ELK lourde.

8.4 Environnements et Gestion des Secrets

La stratégie multi-environnements distingue trois contextes : - **Local / Dev** : Services mockés, bases locales, secrets injectés via fichiers `.env`. - **Staging / Pre-Prod** : Réplique exacte de la production en termes de topologie réseau et Kubernetes, avec données anonymisées ou restreintes. Permet la validation métier finale. - **Production** : Cluster sécurisé, accès restreint. La gestion des secrets (clés d'API OTX, mots de passe de base de données, clés JWT) en production proscrit l'utilisation de variables d'environnement en clair. La solution préconisée est **HashiCorp Vault** (ou AWS Secrets Manager/Azure Key Vault), qui injecte les secrets directement dans les pods Kubernetes au démarrage via un provider CSI, assurant la rotation des secrets et la traçabilité des accès.

9. PLAN D'IMPLÉMENTATION PAR PHASES

Le développement et la mise en production de la plateforme ONYX suivent une méthodologie agile itérative sur un cycle de 12 semaines, divisé en 5 phases critiques.

Phase 0 — Fondations (Semaines 1-2)

Objectifs : Établir l'infrastructure de développement et l'architecture de base (le "Squelette").

Livrables : Monorepo configuré (Turborepo) avec linting/Prettier stricts. Mise en place du pipeline CI (GitHub Actions). Déploiement de la base de données PostgreSQL initiale et du serveur d'authentification (JWT/RBAC). Initialisation du frontend Next.js avec le Design System atomique de base (Couleurs, Typographie, Composants génériques).

Risques : Retard dans la définition du modèle de données de base. *Mitigation :* Adopter le standard STIX 2.1 comme source de vérité structurelle immédiate.

Phase 1 — Modules de Données (Semaines 3-4)

Objectifs : Mettre en œuvre la mécanique d'ingestion et de peuplement du système.

Livrables : Développement des Workers d'ingestion asynchrones. Intégration des flux publics (CISA KEV, Abuse.ch). Implémentation de la déduplication et du scoring de sévérité. Déploiement du microservice Python avec le modèle SciBERT basique opérationnel via API FastAPI. Interface frontend de la "Matrice Géopolitique" connectée à des données d'acteurs de référence statiques.

Risques : Variabilité des formats des feeds externes. *Mitigation :* Créer une couche "Adapter" stricte validant le schéma avant l'ingestion interne.

Phase 2 — Modules Analytiques (Semaines 5-6)

Objectifs : Rendre la donnée ingérée visuellement exploitable pour l'analyste.

Livrables : Déploiement de la base orientée graphe (Neo4j). Développement du composant "Graphe de Menaces" frontend en Cytoscape.js supportant le rendu WebGL et les layouts Force-Directed. Interface de modélisation "Kill Chain" (MITRE ATT&CK) interactive. Création des vues "Fiches Complètes" pour les Threat Actors avec enrichissement des TTPs.

Risques : Performances de rendu du graphe dans le navigateur avec des milliers de nœuds. *Mitigation :* Implémenter un clustering dynamique (Regroupement) et de la pagination de graphe au niveau API.

Phase 3 — Intelligence Avancée et Temps Réel (Semaines 7-8)

Objectifs : Transformer le tableau de bord statique en une plateforme réactive et intelligente.

Livrables : Mise en place du serveur WebSocket et de l'infrastructure Redis Pub/Sub. Intégration de la remontée d'alertes en temps réel sur l'UI (Toasts, mise à jour du Graphe sans rechargement). Finalisation du "Laboratoire IA" avec le scénario déterministe complet (SolarWinds SUNBURST) prêt pour présentation. Fonctionnalité d'export asynchrone des bundles STIX 2.1 validés.

Risques : Surcharge du serveur WebSocket en cas d'afflux massif de nouveaux IoCs. *Mitigation :* Implémentation du "Throttling" et du "Debouncing" des événements côté serveur.

Phase 4 & 5 — Premium, Optimisation, Déploiement (Semaines 9-12)

Objectifs : Polissage final "Executive-Grade", sécurisation et lancement.

Livrables : (S9-10) Création du Tableau de Bord Exécutif (BLUF, métriques macroscopiques). Générateur de rapports dynamiques PDF exécutés via Puppeteer en backend. Campagne d'optimisation des performances (Core Web Vitals cibles atteints, Lazy Loading). (S11-12) Déploiement complet en production via manifests Kubernetes. Configuration des dashboards de monitoring Grafana. Documentation opérationnelle finalisée et remise. Formation de la première cohorte d'utilisateurs SOC.

Risques : Régressions de performance sur l'environnement de production sous charge réelle.

Mitigation : Campagne de tests de charge massifs (K6/JMeter) sur l'environnement de Staging durant la semaine 10.

10. BONNES PRATIQUES ET RECOMMANDATIONS

Conventions de Code et Qualité

Le maintien d'un code de niveau militaire exige une rigueur absolue. L'utilisation d'**ESLint** avec la configuration recommandée de TypeScript et **Prettier** est imposée de force via des hooks `pre-commit` (Husky). Le code non conforme n'entre pas dans le dépôt. Les conventions de nommage TypeScript suivent le standard de l'industrie : `PascalCase` pour les Interfaces, Types et Composants React ; `camelCase` pour les fonctions, variables et instances ; `UPPER_SNAKE_CASE` pour les constantes globales. Aucun type `any` n'est toléré (`eslint: @typescript-eslint/no-explicit-any` réglé sur "error").

Stratégie de Tests

La pyramide de tests vise une couverture de code de 80% des chemins critiques : - **Tests Unitaires (Vitest)** : Valident la logique métier isolée, les fonctions pures (utils), et le reducer Zustand. Exécution rapide en CI. - **Tests d'Intégration (Supertest + Base éphémère)** : Valident les contrats API (Endpoints REST), les insertions en base et l'interaction entre les couches Controller/Service/Repository. - **Tests E2E (Playwright)** : Simulent les flux utilisateurs critiques depuis un navigateur headless (Login, création de rapport, navigation dans le graphe). Limités en nombre pour éviter la fragilité des tests et la lenteur d'exécution CI.

Documentation Systémique

Le code doit être auto-documenté autant que possible par un nommage explicite, cependant des blocs **JSDoc** sont obligatoires pour toutes les fonctions publiques, utilitaires complexes et interfaces. Chaque module principal dans l'arborescence (ex: `src/modules/scibert`) doit contenir un fichier `README.md` expliquant son architecture interne et son comportement. L'API backend expose une documentation interactive vivante générée automatiquement via **OpenAPI 3.0 / Swagger**, accessible sur l'endpoint `/api-docs` en environnement de développement.

Gestion des Erreurs et Résilience

L'utilisation de blocs `try/catch` génériques est proscrite au profit de **classes d'erreurs typées** (ex: `NotFoundError`, `ValidationException`, `ExternalAPIError`). Toutes les erreurs techniques sont interceptées par un middleware central Node.js. Le logger structuré (Pino ou Winston) consigne l'erreur avec la stack trace complète (niveau ERROR). L'API renvoie au client un message d'erreur standardisé, édulcoré (sans exposer l'architecture sous-jacente) et localisé en français.

Performance Frontend

L'interface doit rester fluide même lors de la manipulation de datasets volumineux. Le **Code Splitting** et le **Lazy Loading** sont configurés via `next/dynamic` : le moteur lourd Cytoscape.js n'est chargé que lorsque l'utilisateur accède à la route "Graphe". Les images et assets statiques utilisent le composant natif Next.js `<Image>` pour l'optimisation des formats (WebP/AVIF). Les métriques

"Core Web Vitals" cibles sont strictes : un LCP (Largest Contentful Paint) inférieur à 2.5 secondes et un CLS (Cumulative Layout Shift) proche de 0, garanti par des dimensions explicites sur tous les conteneurs d'interface dynamiques.

11. GLOSSAIRE ET RÉFÉRENCES

Glossaire CTI

- **APT (Advanced Persistent Threat)** : Acteur malveillant souvent étatique, menant des campagnes d'espionnage ou de sabotage ciblées et prolongées.
- **TTP (Tactics, Techniques, and Procedures)** : Description comportementale de la façon dont un adversaire opère. Les TTPs sont le niveau le plus précieux de l'intelligence selon la "Pyramide de la Douleur" de David Bianco.
- **IoC (Indicator of Compromise)** : Empreinte technique laissée par une attaque (adresse IP de C2, hash de fichier malveillant, domaine frauduleux).
- **STIX (Structured Threat Information Expression)** : Langage standardisé (JSON) pour représenter et échanger des informations sur les menaces cybernétiques.
- **TAXII (Trusted Automated Exchange of Intelligence Information)** : Protocole de transport conçu spécifiquement pour l'échange de données STIX sur HTTPS.
- **Kill Chain (Cyber Kill Chain)** : Modèle développé par Lockheed Martin décrivant les phases d'une cyberattaque (Reconnaissance, Weaponization, Delivery, Exploitation, Installation, C2, Actions on Objectives).
- **MITRE ATT&CK** : Base de connaissances globale de tactiques et techniques adversaires, utilisée massivement comme framework de référence pour modéliser les menaces.
- **CVSS (Common Vulnerability Scoring System)** : Standard ouvert pour évaluer la gravité des vulnérabilités logicielles.
- **YARA / Sigma** : Langages de création de règles de détection. YARA pour identifier des patterns dans les fichiers (malwares), Sigma pour les événements de logs (SIEM).
- **C2 / C&C (Command and Control)** : Serveur contrôlé par un attaquant servant à envoyer des commandes aux systèmes compromis et exfiltrer des données.

Références Architecturales et Standards

- OASIS Cyber Threat Intelligence (CTI) Technical Committee - *STIX Version 2.1 Specification*.
- MITRE Corporation - *ATT&CK® Design and Philosophy*.
- Google Research - *SciBERT: A Pretrained Language Model for Scientific Text* (Beltagy et al., 2019).
- Neo4j Graph Database - *Graph Modeling Guidelines for Cyber Security*.
- Next.js Documentation - *App Router & React Server Components Architecture*.
- Agence Nationale de la Sécurité des Systèmes d'Information (ANSSI) - *Guide d'hygiène informatique (2024)*.